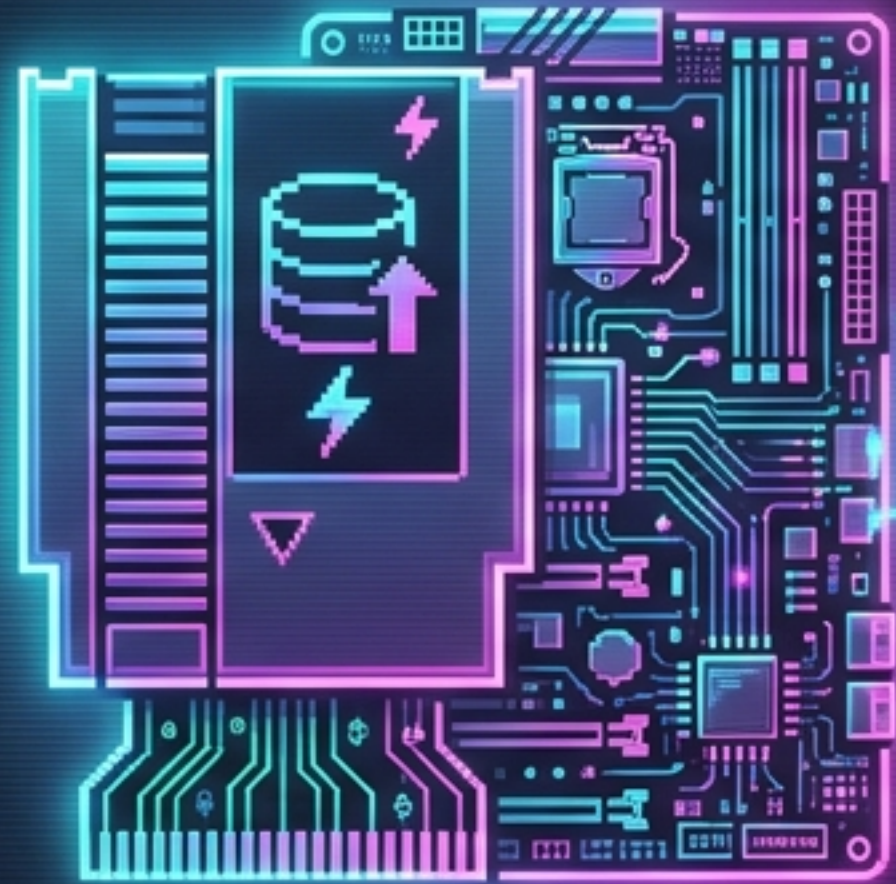




INFORME TÉCNICO 13: La Búsqueda de la Optimización Máxima

Arquitectura, Tuning y Mejores Prácticas T-SQL para DB GradosTitulos

› INSERT COIN TO OPTIMIZE...

SELECT LEVEL: Hoja de Ruta del Sistema



LEVEL 1: XEvents - Proyectos de Monitoreo

[PANEL IZQUIERDO: Proyecto 1 - Consultas de Larga Duración]

1. **Enunciado:** Capturar consultas > 1000 ms en esquemas Académico, Tramite, Evaluacion y Documento.

```
CREATE EVENT SESSION [Monitor_Consultas_Lentas] ON SERVER
  ADD EVENT sqlserver.rpc_completed(ACTION(sqlserver.client_app_name,
sqlserver.database_name, sqlserver.sql_text, sqlserver.plan_handle) WHERE
([duration]>=(1000000) AND [sqlserver].[database_name]=N'GradosTitulos')),
  ADD EVENT sqlserver.sql_batch_completed( /* Mismos parámetros y WHERE
*/ )
  ADD TARGET package0.event_file(SET filename=N'D:\\Logs\\Monitor_Consult
as_Lentas.xel')
  WITH (MAX_MEMORY=4096 KB, EVENT_RETENTION_MODE=ALLOW_SINGLE_EVENT_LOSS,
MAX_DISPATCH_LATENCY=30 SECONDS, STARTUP_STATE=OFF);
GO
ALTER EVENT SESSION [Monitor_Consultas_Lentas] ON SERVER STATE = START;
```

3. Justificación Técnica:

XEvents es más ligero que SQL Profiler (reside en el núcleo). El filtro por predicado evita el Observer Overhead en sistemas concurrentes.

4. Buenas Prácticas:

Aislamiento: Incluir plan_handle para correlación con Plan Cache.
Recolección Asíncrona: Buffer de 4MB minimiza contención.

[PANEL DERECHO: Proyecto 2 - Monitoreo Automático de Salud]

1. **Enunciado:** Capturar consultas lentas, deadlocks, bloqueos, errores > nivel 16 y alto CPU.

```
CREATE EVENT SESSION [Salud_Servidor_GradosTitulos] ON SERVER
  ADD EVENT sqlserver.xml_deadlock_report,
  ADD EVENT
sqlserver.blocked_process_report(ACTION(sqlserver.database_name,
sqlserver.sql_text)),
  ADD EVENT sqlserver.error_reported(ACTION(sqlserver.database_name,
sqlserver.sql_text) WHERE ([severity]>(16))),
  ADD EVENT
sqlserver.scheduler_monitor_system_health_ring_buffer_recorded
  ADD TARGET package0.event_file(SET filename=N'D:\\Logs\\Salud_Servidor
.xel')
```

3. **Justificación Técnica:** xml_deadlock_report genera gráficos de interbloqueo. Errores > 16 previenen fallos antes de una caída.

4. Buenas Prácticas:

Persistencia: Archivos .xel permiten análisis post-mortem de reinicios.

LEVEL 1 BOSS: Estrategia de Diagnóstico Permanente

[Proyecto 3: Línea Base (Baseline)]

1. Enunciado: Diseñar una solución de diagnóstico que consolide métricas de rendimiento y justifique una línea base histórica.

```
CREATE TABLE Tramite.PerformanceBaseline (  
    CaptureDate DATETIME DEFAULT GETDATE(),  
    MetricName VARCHAR(100),  
    MetricValue BIGINT  
);  
GO  
INSERT INTO Tramite.PerformanceBaseline (MetricName, MetricValue)  
SELECT wait_type, waiting_tasks_count  
FROM sys.dm_os_wait_stats  
WHERE wait_type IN ('PAGEIOLATCH_EX', 'SOS_SCHEDULER_YIELD', 'LCK_M_X');
```

3. Justificación Técnica:

Un Baseline permite diferenciar picos estacionales de problemas reales. Sin datos históricos, no se puede validar si un aumento en PAGEIOLATCH_EX es crecimiento natural o código ineficiente.

4. Buenas Prácticas:

Muestreo Intervalar: Programar capturas cada 15 minutos mediante SQL Agent para tener granularidad sin penalizar el rendimiento.





LEVEL 2: Dungeons de Índices - Creación y Fragmentación



[PANEL IZQUIERDO: Proyecto 1 - Consultas de Estudiantes]

1. **Enunciado:** Detectar índices faltantes en Academico y Tramite; crear índice NONCLUSTERED con INCLUDE.

```
-- Detección (Simplificada)
SELECT migs.avg_user_impact, mid.statement,
mid.equality_columns, mid.included_columns
FROM sys.dm_db_missing_index_groups mig JOIN...

-- Implementación
CREATE NONCLUSTERED INDEX IX_Persona_DNI_Optimizado
ON Academico.Persona (DNI)
INCLUDE (Nombres, ApellidoPaterno, ApellidoMaterno, Correo);
```

3. **Justificación Técnica:** Usar INCLUDE elimina el Key Lookup en el Clustered Index, transformando la búsqueda en un Index Seek de solo lectura (reduce CPU y Lecturas Lógicas).

4. Buenas Prácticas:

Selectividad: El DNI es altamente selectivo, siendo el candidato perfecto como clave de búsqueda principal.

[PANEL DERECHO: Proyecto 2 - Corrección de Fragmentación]

1. **Enunciado:** SP que consulte sys.dm_db_index_physical_stats y aplique REORGANIZE (>20%) o REBUILD (>40%).

```
CREATE PROCEDURE Tramite.USP_MantenimientoIndices AS
BEGIN
    SET NOCOUNT ON; DECLARE @SQL NVARCHAR(MAX) = '';
    SELECT @SQL = @SQL + CASE
        WHEN ps.avg_fragmentation_in_percent > 40 THEN 'ALTER INDEX ' +
        QUOTENAME(i.name) + ' ON ' + QUOTENAME(s.name) + ' REBUILD; ' END
        WHEN ps.avg_fragmentation_in_percent > 20 THEN 'ALTER INDEX ' +
        QUOTENAME(i.name) + ' ON ' + QUOTENAME(t.name) + ' REORGANIZE; ' END
    FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL,
    'LIMITED') ps ...
    EXEC sp_executesql @SQL; EXEC sp_updatestats;
END;
```

3. **Justificación Técnica:** REORGANIZE desfragmenta hojas sin bloquear. REBUILD recrea el índice y actualiza estadísticas implícitamente (FULLSCAN).

4. Buenas Prácticas:

Filtro de Tamaño: Omitir mantenimiento en tablas < 1000 páginas; su fragmentación no afecta el rendimiento.

LEVEL 2 BOSS: Automatización Definitiva

[Proyecto 3: Automatización del Mantenimiento]

1. Enunciado: Crear un script para SQL Server Agent que actualice estadísticas y registre la duración en un log.

```
CREATE TABLE Tramite.LogMantenimiento (  
    IdLog INT IDENTITY PRIMARY KEY, FechaInicio DATETIME, FechaFin  
); DATETIME, Tarea VARCHAR(100)  
GO  
DECLARE @Inicio DATETIME = GETDATE();  
-- Tarea de mantenimiento  
UPDATE STATISTICS Tramite.Tramite WITH FULLSCAN;  
UPDATE STATISTICS Academico.Persona WITH FULLSCAN;  
  
INSERT INTO Tramite.LogMantenimiento (FechaInicio, FechaFin, Tarea)  
VALUES (@Inicio, GETDATE(), 'Mantenimiento Semanal de Estadísticas');
```

3. Justificación Técnica:

Estadísticas desactualizadas generan planes de ejecución subóptimos. FULLSCAN garantiza 100% de precisión en el histograma, permitiendo al optimizador decidir correctamente entre un Nested Loop o un Hash Join.

4. Buenas Prácticas:

Ventanas de Mantenimiento: Ejecutar en horarios de mínima carga (Safe Zones) para evitar contención por bloqueos de esquema (Sch-M).





LEVEL 3: Escudos ACID - Control y Bloqueos



[PANEL IZQUIERDO: Proyecto 1 - Aprobación de Títulos]

1. Enunciado: Script con BEGIN TRAN y TRY...CATCH afectando múltiples tablas de trámites.

```
BEGIN TRY
  BEGIN TRANSACTION;
  UPDATE Tramite.Tramite SET IdEstadoTramite = 5 WHERE IdTramite = 101;
  INSERT INTO Documento.Resolucion (...) VALUES (...);
  INSERT INTO Documento.Diploma (...) VALUES (...);
  UPDATE Evaluacion.EvaluacionTrabajo SET Condicion = 'APROBADO' WHERE
  IdTrabajo = 50;
  COMMIT TRANSACTION;
END TRY
BEGIN CATCH
  IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
  PRINT 'Error: ' + ERROR_MESSAGE();
END CATCH;
```

3. Justificación Técnica: Garantiza Atomicidad (ACID). ROLLBACK evita registros huérfanos o estados inconsistentes si falla la inserción del Diploma.

4. Buenas Prácticas:

Transacciones Cortas: Procesar solo lo estrictamente necesario dentro del bloque para reducir el tiempo de retención de Locks.

[PANEL DERECHO: Proyecto 2 - Resolución de Bloqueos]

1. Enunciado: Generar bloqueo y detectarlo mediante DMVs identificando la sesión raíz.

```
-- Sesión 1 (Generador): UPDATE sin COMMIT

-- Sesión 2 (Detección):
SELECT r.session_id AS Sesion_Bloqueada, r.blocking_session_id AS
Sesion_Bloqueadora,
       r.wait_type, r.wait_time, t.text AS Consulta_Bloqueada
FROM sys.dm_exec_requests r CROSS APPLY sys.dm_exec_sql_text
(r.sql_handle) t
WHERE r.blocking_session_id <> 0;
```

3. Justificación Técnica: Monitorear blocking_session_id halla la raíz de la degradación. Es la respuesta del motor para proteger la integridad (Pessimistic Concurrency).

4. Buenas Prácticas:

Monitoreo Activo: Usar DMVs para aislar sesiones huérfanas antes de desencadenar un efecto dominó.

LEVEL 3 BOSS: Prevención de Deadlocks (Abrazo Mortal)

[Proyecto 3: Laboratorio de Deadlocks]

- 1. Enunciado: Escenario de acceso inverso a Academico.Boleto Academico.Docente y Tramite.DesignacionAsesor y su rediseño.

```
1 -- Proceso de Designación Estandarizado (El orden importa)
2 BEGIN TRANSACTION;
3
4 -- 1. Siempre acceder primero a la tabla de mayor jerarquía o padre
5 UPDATE Academico.Docente SET EstadoActivo = 1 WHERE IdDocente = 5;
6
7 -- 2. Luego acceder a la tabla dependiente
8 INSERT INTO Tramite.DesignacionAsesor (IdTrabajo, IdDocente, NroResolucion)
9 VALUES (10, 5, 'RES-001');
10
11 COMMIT TRANSACTION;
```

3. Justificación Técnica:

Los deadlocks nacen por bloqueos cruzados (procesos que solicitan recursos en orden inverso). Estandarizar el orden (A -> B siempre) transforma el ciclo mortal en una simple cola de espera lineal (Queue).

4. Buenas Prácticas:

Uso de Índices: Los deadlocks suelen ser causados por Scans que bloquean páginas enteras; forzar Seeks mediante índices reduce la superficie de bloqueo a nivel de fila.

LEVEL 4: El Oráculo - Planes de Ejecución

[PANEL IZQUIERDO: Proyecto 1 - Diagnóstico de Consultas]

1. **Enunciado:** Analizar consulta de 5 JOINS (Auditoría Expediente) e identificar operadores costosos.

```
SELECT p.Nombres, m.FechaMatricula, t.IdTramite, ti.Titulo, r.NroResolucion  
FROM Academico.Persona p INNER JOIN Academico.Matricula m ...  
WHERE p.DNI = '12345678';
```



3. **Justificación Técnica:** Table Scan lee millones de filas. Hash Match consume alta RAM (falta índice FK). Key Lookup genera I/O adicional.

4. **Buenas Prácticas: Modelado Relacional:** Indexar llaves foráneas para habilitar Merge Joins en lugar de Hash Matches.

[PANEL DERECHO: Proyecto 2 - Impacto en TrabajoInvestigacion]

1. **Enunciado:** Comparativa Antes/Después al indexar TipoTrabajo.

```
CREATE NONCLUSTERED INDEX IX_Trabajo_Tipo_Busqueda  
ON Tramite.TrabajoInvestigacion (TipoTrabajo) INCLUDE (Titulo);
```

Métricas de Impacto Matemático:

Logical Reads	Antes: 1,250	Después: 4	Mejora: 99.6%
Estimated Cost	Antes: 0.450	Después: 0.003	Mejora: 99.3%

3. **Justificación Técnica:** El plan de ejecución pasó de un costoso Scan a un eficiente Seek.

4. **Buenas Prácticas: Cobertura:** Incluir (INCLUDE) las columnas del SELECT evita que el motor regrese a la tabla base (elimina Key Lookups).

LEVEL 4 BOSS: Derrotando el Parameter Sniffing

[Proyecto 3: Optimización de SP de Resoluciones]

1. **Enunciado:** Aplicar técnicas para mejorar reutilización del plan y eliminar Index Scans por sesgo de parámetros.

```
CREATE OR ALTER PROCEDURE Documento.USD_ObtenerResolucion
    @NroResolucion VARCHAR(50)
AS
BEGIN
    SET NOCOUNT ON;
    -- Uso de variable local para mitigar Parameter Sniffing (Enmascaramiento)
    DECLARE @NroResLocal VARCHAR(50) = @NroResolucion;

    SELECT IdTramite, FechaEmision, OrganoEmisor
    FROM Documento.Resolucion
    WHERE NroResolucion = @NroResLocal;
END;
```



PARAMETER SNIFFING CONTAINED

3. Justificación Técnica:

Al asignar el parámetro a una variable local, el optimizador no puede husmear el valor exacto en tiempo de compilación. Genera un plan basado en la densidad promedio (comportamiento similar a OPTIMIZE FOR UNKNOWN), garantizando un rendimiento estable para todas las llamadas.

4. Buenas Prácticas:

Estabilidad sobre Picos:
Es preferible tener un plan suficientemente bueno siempre, a tener un plan excelente para un valor y valor y desastroso para el resto debido a la asimetría de los datos.

LEVEL 5: Magia Avanzada - Reescritura T-SQL



[PANEL IZQUIERDO: Proyecto 1 - Consultas de Historial]

1. **Enunciado:** Eliminar subconsultas en historial usando CTE y lógica EXISTS.

```
WITH Historial_CTE AS (  
    SELECT t.IdTramite, p.DNI, et.Descripcion AS EstadoActual  
    FROM Tramite.Tramite t /* ... JOINS ... */  
)  
  
SELECT * FROM Historial_CTE h  
WHERE EXISTS (SELECT 1 FROM Documento.Diploma d  
    WHERE d.IdTramite = h.IdTramite);
```

3. **Justificación Técnica:** EXISTS es un Semi-Join. Se detiene al hallar la primera coincidencia, evitando el costoso producto cartesiano masivo de un JOIN tradicional.

4. **Buenas Prácticas:** Legibilidad: Usar CTEs modulariza la lógica de negocio sin penalizar el rendimiento del optimizador.



[PANEL DERECHO: Proyecto 2 - Reportes Académicos]

1. **Enunciado:** Uso de funciones ventana para obtener el último estado del trámite.

```
SELECT IdMatricula, IdEstadoTramite, FechaUltMod,  
    ROW_NUMBER() OVER(PARTITION BY IdMatricula ORDER BY  
        FechaUltMod DESC) AS Secuencia  
FROM Tramite.Tramite  
WHERE FechaInicio >= '2026-01-01'; -- SARGable early filter
```

3. **Justificación Técnica:** El filtrado temprano (WHERE) reduce el conjunto de datos antes de que ocurra el particionamiento y ordenamiento (Sort) de la función ventana.

4. **Buenas Prácticas:** Filtros SARGable: Aplicar filtros eficientes antes de funciones ventana ahorra ciclos críticos de CPU y RAM.

LEVEL 5 BOSS: Erradicación de Cursores

[Proyecto 3: Búsqueda de Expedientes (Lógica Set-Based)]

1. Enunciado: Sustituir ineficientes cursores por lógica basada en conjuntos con Recompile.

Syntax

```
-- DESPUÉS: Lógica basada en conjuntos (Set-Based) con Hint
CREATE OR ALTER PROCEDURE Tramite.USP_ActualizarObservacionesMasivas
    @IdCoordinacion TINYINT
AS
BEGIN
    UPDATE Tramite.Tramite
    SET Observaciones = 'REVISIÓN ANUAL 2026'
    WHERE IdCoordinacion = @IdCoordinacion
    OPTION (RECOMPILE); -- Solución forzada para sesgo de datos atípico
END;
```

3. Justificación Técnica:

El enfoque Set-based permite al motor usar procesamiento en paralelo. Un cursor obliga a leer disco y bloquear fila a fila (RBAR); la actualización masiva gestiona la transacción en un solo bloque eficiente.

4. Buenas Prácticas:

Uso Selectivo de Hints: `OPTION (RECOMPILE)` se justifica solo si la distribución de datos es drásticamente desigual (ej. un Dpto tiene 10 trámites, otro tiene 100,000) y un solo plan cacheado resultaría desastroso.



PARALLEL BATCH
PROCESSING

LEVEL 6: El Gobernador - Resource Governor

MAX CAP (LIMITATION)

30%

[PANEL IZQUIERDO: Proyecto 1 - Carga Administrativa]

1. **Enunciado:** Crear Pool/Group para limitar a administrativos a un 30% de CPU.

```
CREATE RESOURCE POOL Pool_Administrativo WITH (MAX_CPU_PERCENT = 30);
CREATE WORKLOAD GROUP Group_Administrativo USING Pool_Administrativo;
GO
CREATE FUNCTION dbo.UDF_Classifier_Grados() RETURNS SYSNAME WITH SCHEMABINDING AS
BEGIN
    IF (SUSER_SNAME() = 'user_admin_report') RETURN 'Group_Administrativo';
    RETURN 'default';
END;
GO
ALTER RESOURCE GOVERNOR WITH (CLASSIFIER_FUNCTION = dbo.UDF_Classifier_Grados);
ALTER RESOURCE GOVERNOR RECONFIGURE;
```

3. **Justificación Técnica:** Protege el motor aislando el impacto de reportes masivos (MAX 30%), asegurando CPU para operaciones transaccionales críticas.

4. **Buenas Prácticas:** Clasificador Eficiente: La UDF de clasificación debe ser extremadamente ligera; complejidad aquí degrada cada conexión entrante.

MIN RESERVE (GUARANTEE)

50%

[PANEL DERECHO: Proyecto 2 - Priorización Crítica]

1. **Enunciado:** Prioridad alta a procesos de emisión en Cierre Semestral.

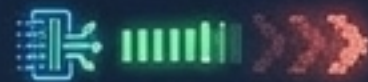
```
CREATE RESOURCE POOL Pool_Critico WITH (MIN_CPU_PERCENT = 50);
CREATE WORKLOAD GROUP Group_Diplomas WITH (IMPORTANCE = HIGH)
    USING Pool_Critico;
GO
ALTER RESOURCE GOVERNOR RECONFIGURE;
```

3. **Justificación Técnica:** IMPORTANCE = HIGH obliga al planificador a adelantar los hilos de este grupo en presión de CPU.

4. **Buenas Prácticas:** Reserva de Recursos: MIN_CPU_PERCENT garantiza una cuota base innegociable incluso si el servidor colapsa por otras consultas.

FINAL BOSS: Monitoreo Activo del Consumo (El Ojo Que Todo lo Ve)

[Proyecto 3: Monitoreo de Recursos en Tiempo Real]



1. Enunciado: Desarrollar un script de consulta sobre DMVs para reportar consumo de Pools y Groups en vivo.

Syntax

```
SELECT
  g.name AS Workload_Group,
  p.name AS Resource_Pool,
  g.total_cpu_usage_ms,
  g.active_request_count,
  g.max_memory_kb / 1024 AS Max_Memory_MB
FROM sys.dm_resource_governor_workload_groups g
JOIN sys.dm_resource_governor_resource_pools p ON g.pool_id = p.pool_id;
```

3. Justificación Técnica:

Asignar límites a ciegas es peligroso. Este monitoreo valida matemáticamente si las cuotas (MAX/MIN_CPU) están asfixiando los procesos o si se están respetando. Es la herramienta definitiva de calibración.

4. Buenas Prácticas:

Ajuste Iterativo: El Resource Governor no se configura una vez y se olvida. Se deben analizar estas DMVs periódicamente para re-balancear los Resource Pools según evoluciona el uso real de producción.

SYSTEM OPTIMIZED: GradosTitulos Database

SCORE PERFECTO EN RENDIMIENTO:

- Latencia Destruida: Operaciones Set-Based, Cobertura de Índices y Filtrado SARGable.
- Concurrencia Dominada: Control Transaccional ACID estricto y Deadlocks Mitigados por jerarquía.
- Escalabilidad Asegurada: Pools de Resource Governor calibrados y Diagnósticos XEvents silenciosos.

> LOGOUT_